

Decision Trees

Dr. Krishna Bathula

Learning Objectives

1. Decision Trees
2. Workflow
3. Feature Selection
4. Random Forests
5. Hyperparameters

Introduction

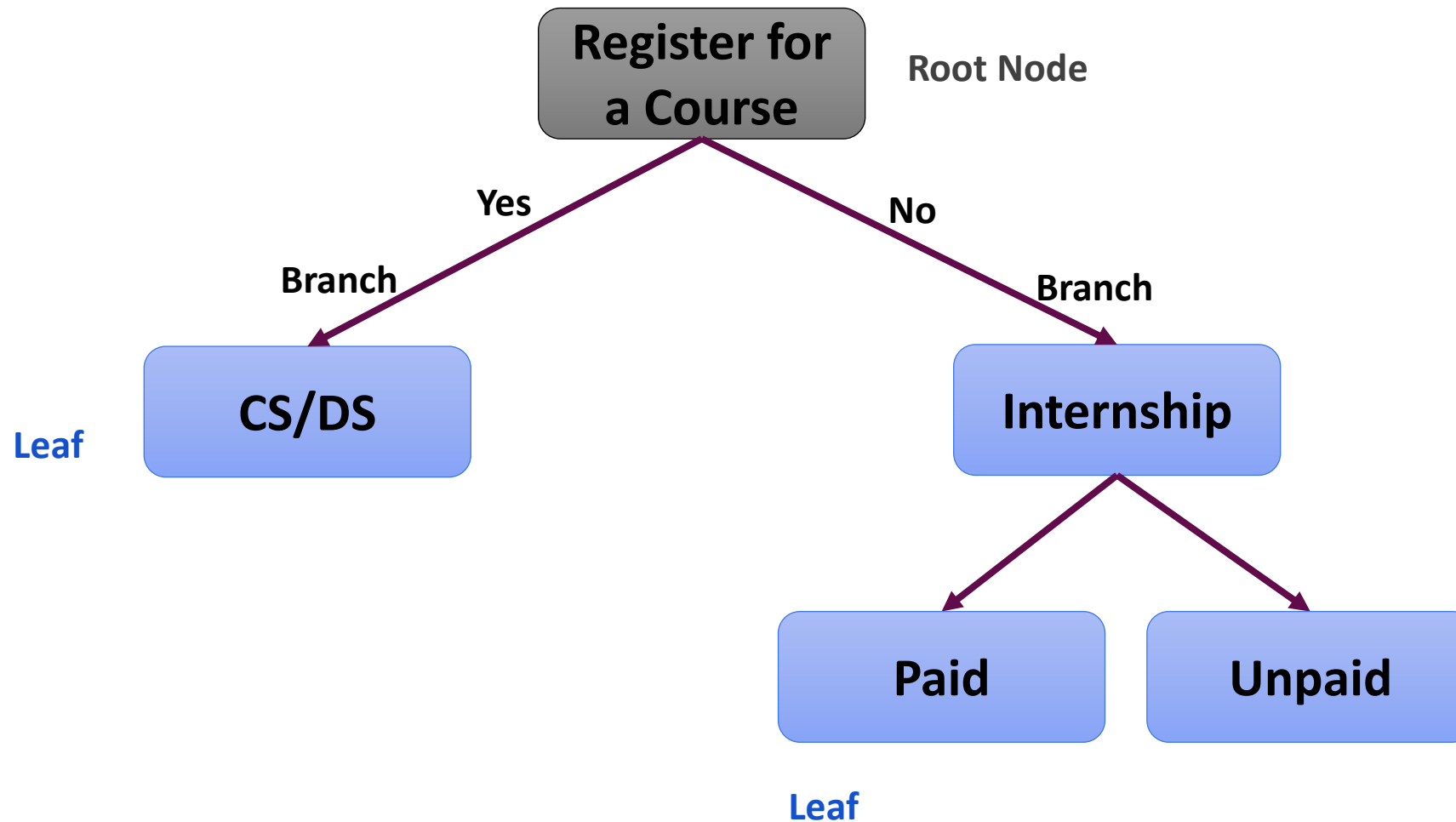
Decision trees are used to classify or predict a target variable based on a set of input features.

They can be used for both **regression** and **classification** tasks.

A decision tree is a tree-like model where each internal node represents a test on a **feature** or attribute.

A decision tree can be represented graphically where each node represents a **decision** or a **test condition**, and the branches represent the outcomes of the test.

Decision Tree Structure



Decision Tree Types

Decision trees used in Machine Learning are of two main types:

Classification tree analysis is when the predicted outcome is the class (discrete) to which the data belongs. (e.g. Rainy - Sunny)

Regression tree analysis is when the predicted outcome can be considered a real number (e.g. the price of a house)

Classification And Regression Tree (CART) analysis refers to either of the above procedures, first introduced by Breiman et al. in 1984.

Trees used for regression and trees used for classification have some similarities and differences, such as the procedure used to determine where to split.

Workflow

The algorithm uses a **recursive approach** to build a decision tree, starting with the **entire dataset** and recursively splitting it into **subsets** based on the **feature**.

Once the tree is built, it can be used to make predictions on new data by **traversing the tree** from the root node down to a leaf node.

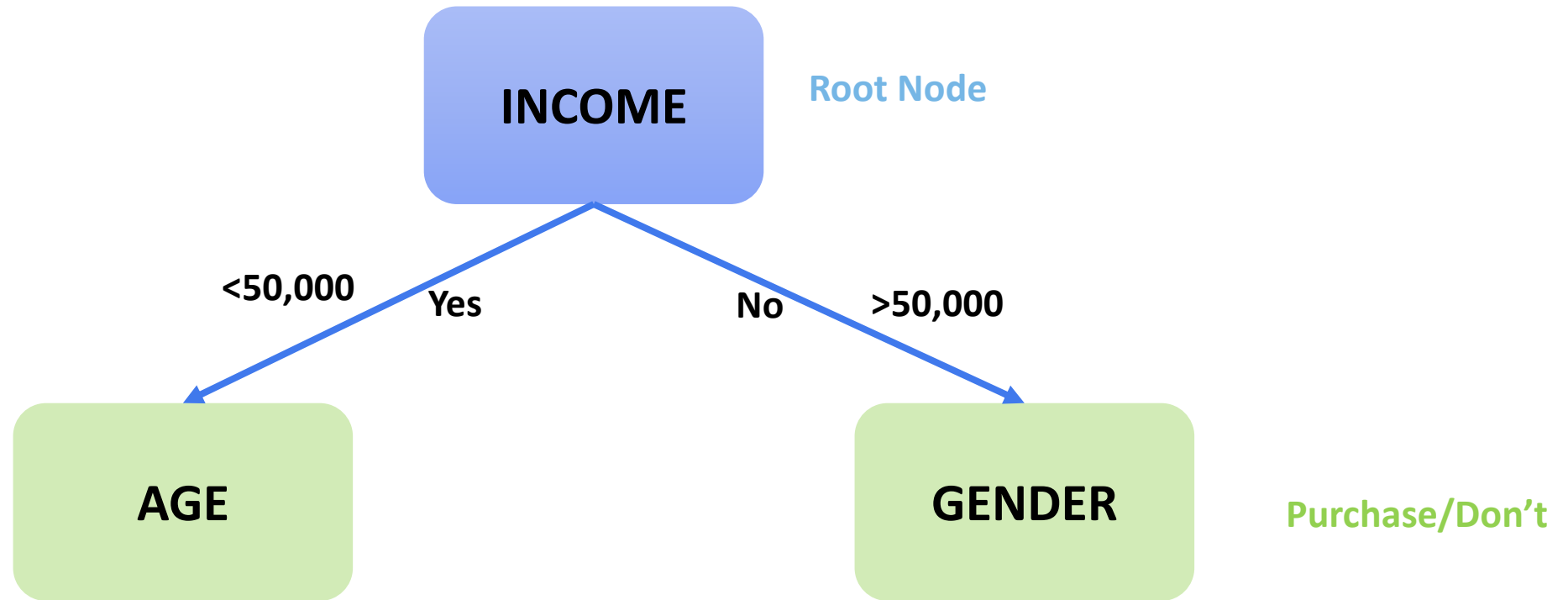
At each internal node, the algorithm tests the value of a **feature** or attribute and follows the **corresponding branch** based on the outcome of the test.

The process continues until a leaf node is reached, which represents a **class label** or a **numerical value**.

Decision Tree Predictions

CUSTOMER

- AGE
- INCOME
- GENDER



Feature Importance Analysis

Feature importance analysis refers to a class of techniques for assigning scores to input features to a predictive model that indicates the relative importance of each feature when making a prediction.

The scores are useful for better understanding the data, model and thereby reducing the number of input features.

Decision tree algorithms like classification and regression trees (CART) offer importance scores based on the reduction in the criterion used to select split points, like **Gini Impurity** or **Entropy**.

This same approach can be used for ensembles of decision trees, such as the random forest.

Gini Impurity

In decision trees, Gini Impurity is a metric used to measure the impurity or randomness of a particular node in the tree.

Gini Impurity is calculated by **subtracting the sum of the squared probabilities** of each class from 1.0.

It measures the **probability of incorrectly classifying** a randomly chosen element in the dataset if it were randomly labeled according to the distribution of labels in the node.

When building a decision tree, the algorithm will use Gini Impurity to decide which feature to split the data on at each node.

Gini Impurity

First, it selects the feature that results in the **greatest reduction of Gini Impurity** between the parent node and the child nodes.

The formula for Gini Impurity is:

$$\text{Gini Impurity} = 1 - \sum (\text{probability of each class})^2 \quad G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

Where $\sum$ represents the **sum of all classes**.

The resulting value of Gini Impurity **ranges from 0 to 1**, with 0 indicating a completely pure node with all elements belonging to the same class, and 1 indicating a completely impure node with an equal number of elements belonging to each class.

Entropy

The expected volume of “information,” “surprise,” or “uncertainty” associated with a randomly chosen variable’s potential outcomes is characterized as the entropy of the variable in information theory.

Entropy helps us to build an appropriate decision tree for selecting the best splitter.

Entropy can be defined as a measure of the purity of the sub-split.

Entropy always lies between 0 to 1. The entropy of any split can be calculated by

$$H(s) = -P_{(+)} \log_2 P_{(+)} - P_{(-)} \log_2 P_{(-)}$$

Example

Start by building the root node of the decision tree.

Use the **Gini impurity** or **Entropy** to determine how to split the data first.

Information Gain can then select an important feature to split.

Suppose we find that the **income feature** provides the most **information gain**.

We split the data into two subsets: one with **income less than** or equal to a certain **threshold**, and the other with income **greater than the threshold**.

Then create two child nodes, one for each subset.

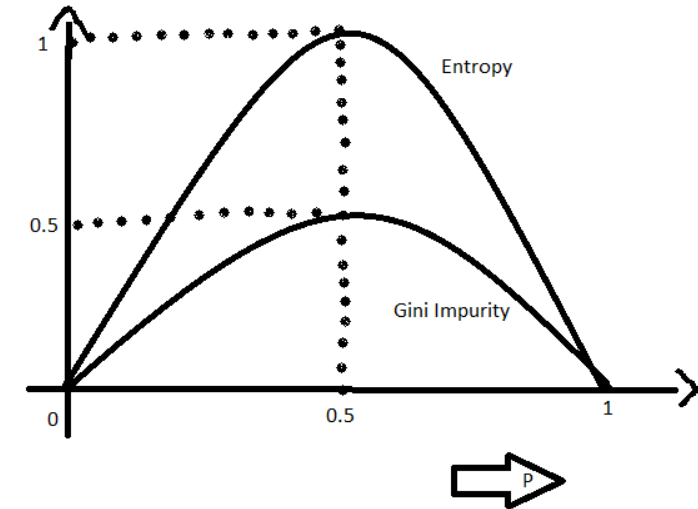
For each child node, repeat the process of determining the feature that provides the most information gain and splitting the data on that feature.

Entropy vs Gini Impurity

Gini Impurity is generally more computationally efficient than entropy.

The graph of entropy increases up to 1 and then starts decreasing, while Gini Impurity only goes up to 0.5 before decreasing, thus requiring less computational power.

The range of entropy is from 0 to $(\log_2 C)$, whereas the range of Gini Impurity is from 0 to 0.5 (for binary classification).



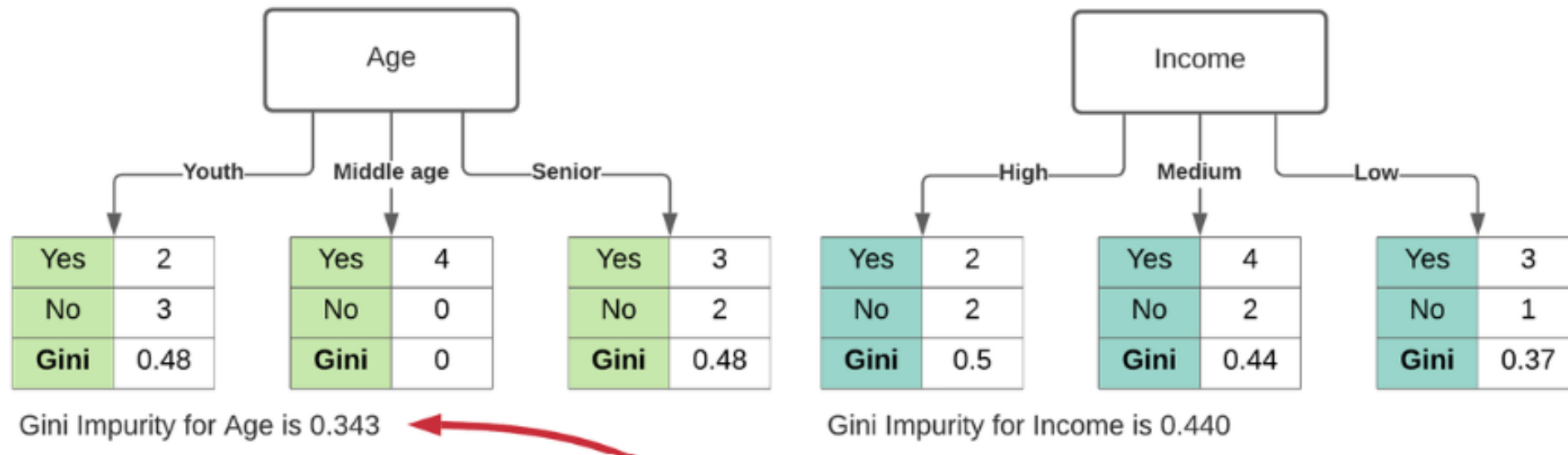
$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

Gini Impurity or Entropy?

By default, the Gini impurity measure is used. We can select the entropy impurity measure instead by setting the criterion **hyperparameter** to "**entropy**".

- Both lead to similar trees most of the time.
- Gini impurity is slightly faster to compute, so it is a good default.
- Entropy tends to produce slightly more balanced trees.



Feature Selection

Information gain determines the best attribute or feature to split the data.

The algorithms recursively split a node it into subsets based on the feature that provides the most information gain or the least impurity.

The goal of the DT is to create a tree that splits the data into subsets that are as homogeneous as possible in terms of the target variable.

The algorithm tests the value of a feature or attribute and follows the corresponding branch

Information Gain

Information gain is a metric used in decision trees to measure the usefulness of a particular feature in predicting the class labels of data.

It is based on the concept of **entropy**, which measures the amount of uncertainty or randomness in a dataset.

Information gain measures the **reduction in entropy** achieved by splitting the dataset on a particular feature.

The **higher the information gain**, the **more effective the feature** is at reducing the uncertainty in the dataset and improving the accuracy of the resulting decision tree.

The formula for information gain is:

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - [\text{Weighted Average}] * \text{Entropy}(\text{children})$$

Information Gain

Where:

Entropy(parent) is the entropy of the parent node, which is calculated based on the distribution of class labels in the entire dataset.

Entropy(children) is the entropy of the child nodes, which are created by splitting the dataset on the selected feature.

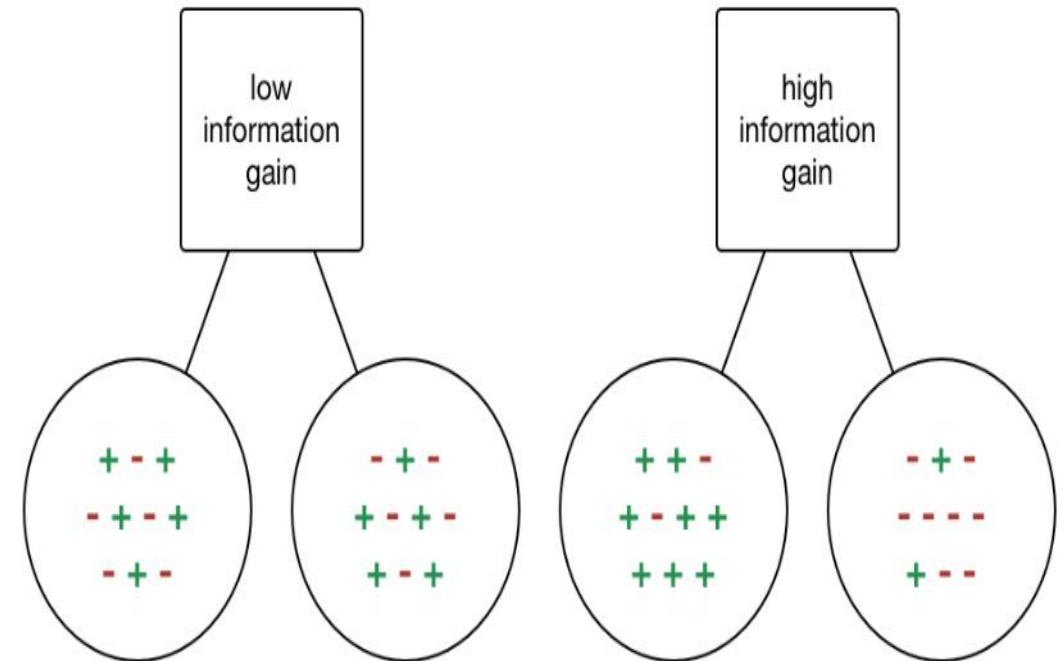
[Weighted Average] is the fraction of the total number of instances that belong to each child node, weighted by the size of the child node.

Information Gain

Information gain calculates the difference between the entropy of the **parent node** and the **weighted average of the entropy of the child nodes**.

The feature with the **highest information gain** is chosen as the next split in the decision tree.

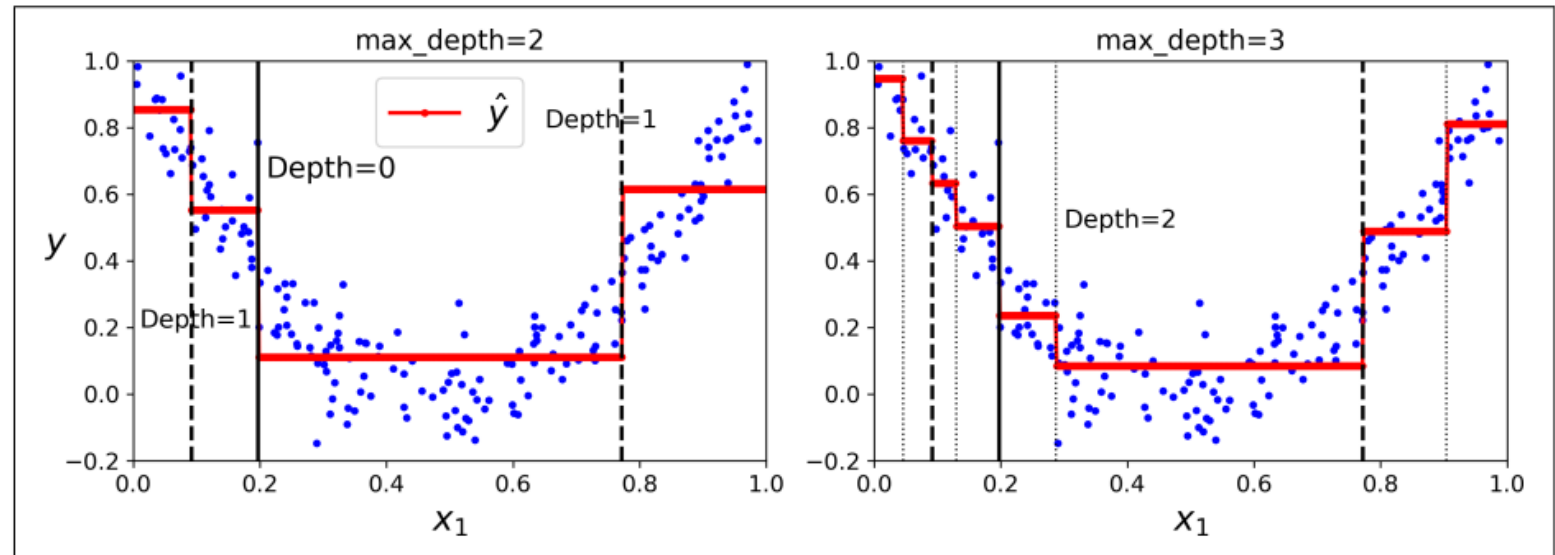
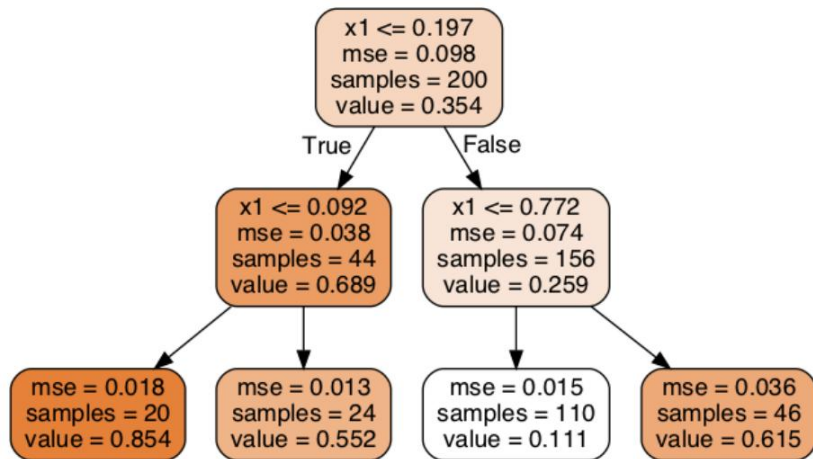
Overall, information gain plays an important role in decision tree algorithms as it helps to identify the best features to split on and build a more accurate model.



DT-Regression Problem

Instead of predicting a class in each node, it predicts a value.

The algorithm splits each sample subset to make most training instances as close as possible to that predicted value.

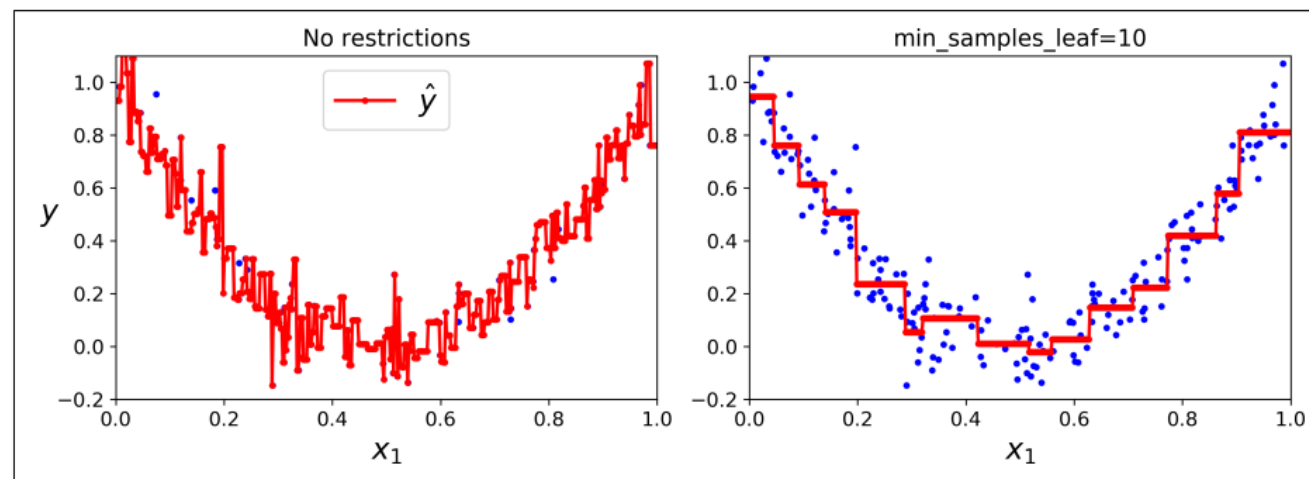


DT-Regression Problem with Regularizing

Decision Trees are prone to **overfitting** when dealing with regression tasks.

We get the predictions on the left without any regularization (i.e., using the default hyperparameters). It is overfitting the training set very badly.

There is a setting `min_samples_leaf=10`. It results in a much more reasonable model, represented on the right



DT- Advantages and Disadvantages

Advantages:

1. Easy to interpret and understand,
2. Handle both categorical and numerical data,
3. Handle missing values and outliers, and
4. Feature selection.

Disadvantages:

1. Prone to overfitting,
2. Sensitive to small variations in the data, and
3. Does not capture complex relationships in the data.

Pruning

Pruning is a technique used in decision trees to **reduce overfitting** and improve the model's generalization performance.

Overfitting occurs when the **decision tree is too complex** and fits the training data too closely, resulting in poor performance on new data.

Pruning involves removing branches or nodes from the decision tree to simplify it and make it more generalizable.

Pre-pruning involves **stopping the tree building process early**, before the tree becomes too complex.

This is done by setting a **limit on the depth of the tree**, the **number of leaf nodes**, or the number of instances required to split a node.

Pruning - Balance DT

For example, we may stop splitting a node if the number of instances in the subset is below a certain **threshold**.

This **reduces the size of the tree** and prevents overfitting.

Pruning can be a useful technique for improving the performance of decision trees, especially when the **data is noisy** or the **features are irrelevant**.

However, pruning can also result in **underfitting if too many nodes or branches are removed**.

The key is to find the right balance between complexity and simplicity to achieve the best performance on new data.

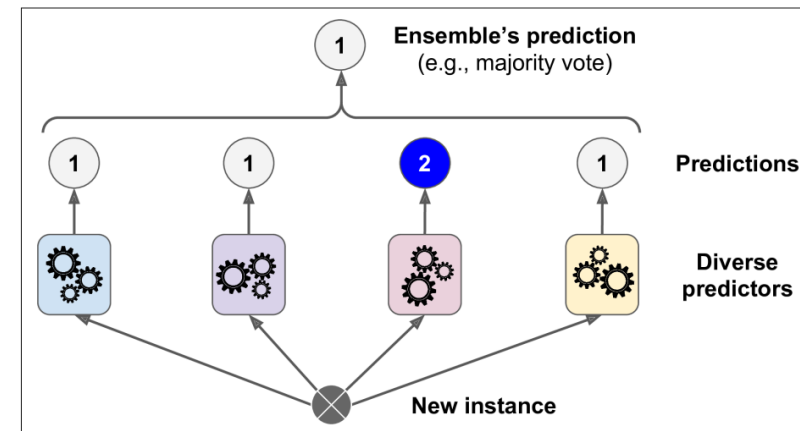
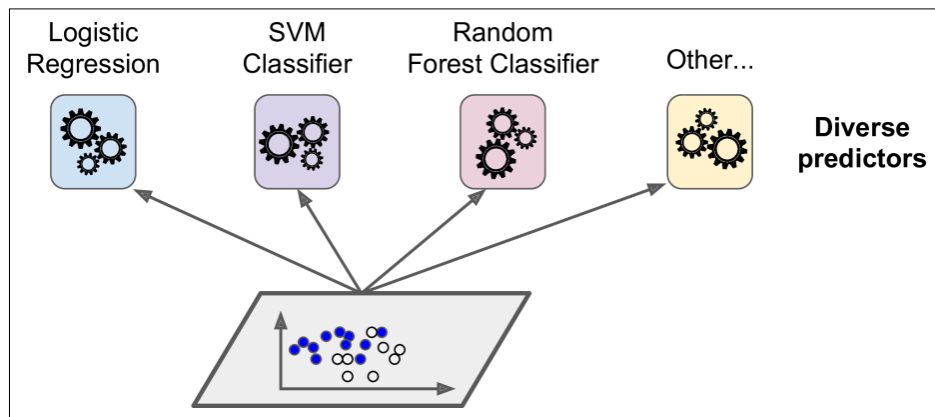
Ensemble Learning

A group of predictors is called an **Ensemble**.

When we aggregate the predictions of a group of predictors (such as classifiers or regressors), we often get better predictions than the best individual predictor.

This technique is called **Ensemble Learning**.

An ensemble Learning algorithm is called an Ensemble method.

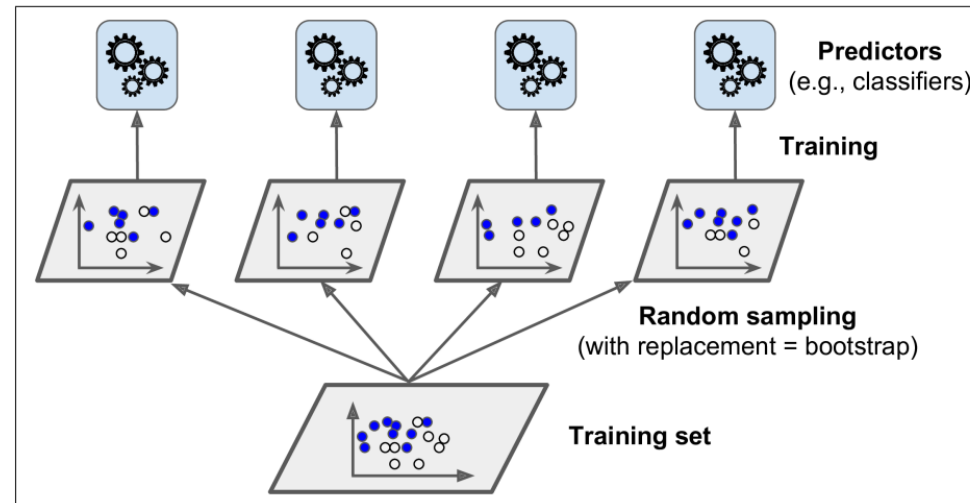


Bagging

One way to get a **diverse set of classifiers** is to use very different training algorithms.

Another approach is to use the same training algorithm for every predictor, but to train them on different **random subsets of the training set**.

When **sampling is performed with replacement**, the process is called bagging.



Random Forest

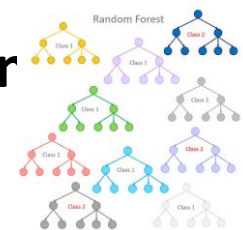
Random forests are build on the concept of decision trees.

Random forests are an ensemble method that combines **multiple decision trees** to improve the accuracy and robustness of the model.

The main idea behind random forests is to **build a large number of decision trees** and then **aggregate the predictions** of all the trees to produce a final prediction.

Each decision tree is trained on a **different subset of the data**, and at each node of the tree, a random subset of the features is considered for splitting the data.

This random selection of features **helps to reduce the correlation between** trees and reduces the risk of overfitting.



Random Forest - Hyperparameters

The random forest algorithm has several hyperparameters that can be tuned to optimize the performance of the model.

Hyperparameters such as the **number of trees**, the **depth of the trees**, and the **number of features** considered at each node can be selected.

Random forests have several advantages over decision trees, such as

- Less prone to overfitting,
- Handle high-dimensional data, and
- More robust to noise and outliers.

They are often used as benchmark algorithms in machine learning competitions.

Random Forest as Ensemble

A Random Forest is an ensemble of Decision Trees, generally trained via the bagging method

The Random Forest algorithm introduces **extra randomness when growing trees**; instead of searching for the very best feature when splitting a node.

In a random forest, **N** decision trees are trained on each one on a **subset of the original training set** obtained via **bootstrapping** of the original dataset, i.e., via random sampling with replacement.

RF searches for the **best feature** among a random subset of features.

This results in a greater tree diversity, and trades a higher bias for a lower variance, generally yielding an overall better model.

Majority Rule

The **N** slightly **differently trained trees** will produce **N** slightly **different predictions** for the same input vector.

The **majority rule** is applied to make the final decision. The prediction offered by the **majority of the N trees** is adopted as the final one.

The advantage of such a strategy is that the predictions from a single tree are highly **sensitive to noise** in the training set, predictions from the majority of many trees are not.

We assume that the trees are **not correlated**.

If the trees are correlated then **Bootstrap sampling** is the way to decorrelate the trees by training them on different training sets.

Out-Of-Bag

A popular **metric** to measure the **prediction error of a random forest** is the out-of-bag error.

Out-of-bag error is the **average prediction error** calculated on all training samples x_i , using only the trees that did not have x_i in their bootstrapped training set.

Out-of-bag error estimates avoid the need for an independent validation dataset but might underestimate actual performance improvement.

XGBoost

XGBoost (Extreme Gradient Boosting) is a popular machine learning algorithm that is used for classification and regression tasks.

It is a variant of the gradient boosting algorithm that uses a more regularized model to prevent overfitting.

The key idea behind XGBoost is to combine multiple weak learners, typically decision trees, to form a stronger learner.

In each iteration of the algorithm, a new decision tree is trained to correct the errors of the previous tree.

The output of the trees is then combined to make the final prediction.

XGBoost

The learning process is guided by an objective function that measures the performance of the model on the training data.

XGBoost differs from traditional gradient boosting algorithms in several ways:

1. **Regularization:** XGBoost includes both L1 and L2 regularization terms in the objective function to prevent overfitting. This helps reduce the **model's variance** and improve its **generalization** performance.
2. **Tree Pruning:** XGBoost uses the pruning technique to remove branches that contribute little to the model's overall accuracy. This helps reduce the **model's complexity** and improve its computational efficiency.

XGBoost

3. **Parallelization:** XGBoost supports parallel processing on a single machine, allowing it to train large datasets much faster than traditional gradient boosting algorithms.

4. **Customizable Loss Functions:** XGBoost allows users to define their own loss functions to optimize the model for specific tasks.

END
